



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

Parche de ejecutable

P R E S E N T A N:

Daniel Olvera Romero - 22590344

Luis Fernando Loma Navarrete - 22590333

Jose Pedro Cruz Ramírez - 22590322

Ingeniería en Sistemas Computacionales

Periodo: Agosto - Diciembre 2026



Tabla de contenido

1. Objetivo.....	4
2. Alcance y consideraciones éticas.....	4
3. Herramientas y archivos utilizados.....	4
4. Descripción general del escenario	5
5. Análisis dinámico con x32dbg.....	5
5.1 Carga del ejecutable y ubicación del código de interés	5
5.2 Breakpoint y seguimiento del flujo.....	6
5.3 Localización de la instrucción decisiva	9
6. El parche	10
7. Automatización del parche en Python	12
8. Pruebas funcionales	12
9. Resultados	13
10. Conclusiones.....	14



Índice de figuras

<i>Ventana del programa Hackeame y CPU de x32dbg justo después de cargar phackeame.exe y dejarlo correr.</i>	5
<i>Listado completo de referencias de cadenas de texto (Strings) del módulo phackeame.exe.</i>	6
<i>Referencia localizada al filtrar por "USR": la cadena {"USR":} se usa en la dirección 0042A1A4.</i>	6
<i>Código en 0042A1A4: construcción del cuerpo JSON con usuario y contraseña, envío por POST y verificación de conexión con el servidor.</i>	7
<i>Breakpoint disparado en 0042A1A4 al introducir el usuario "Ladrillo" y presionar "Acceso".</i>	8
<i>Procesamiento de la respuesta del servidor ({"R":400}) recibida tras enviar una credencial incorrecta.</i>	8
<i>Instrucción decisiva localizada en 0042A25F-0042A266: comparación del StatusCode contra 0xC8 (200) y el salto JNE.</i>	9
<i>Menú contextual Binary -> Fill with NOPs, usado para neutralizar el salto condicional.</i>	10
<i>Resultado del parche en memoria: 0042A266 y 0042A267 convertidas en nop, sin alterar el resto de la función.</i>	11
<i>Ventana de Patches confirmando "2/2 patch(es) applied!": únicamente 2 bytes modificados (75->90, 1C->90).</i>	12
<i>Prueba funcional: phackeame_parcheDg.exe concede acceso ("Lo lograste!") con la credencial inválida "Ladrillo".</i>	13



1. Objetivo

Analizar phackeame.exe (ejecutable compilado en Lazarus/Free Pascal que valida usuario y contraseña contra un servicio web) con x32dbg, localizar la instrucción que decide si el acceso se concede o se rechaza, y modificarla para que el programa conceda acceso sin importar el usuario y la contraseña introducidos. Adicionalmente, automatizar el parche con un script en Python que genere una copia modificada a partir del ejecutable original.

2. Alcance y consideraciones éticas

La práctica se realizó únicamente sobre el ejecutable de laboratorio proporcionado por el docente para esta asignatura. El propósito es estrictamente académico: comprender cómo una decisión de acceso codificada como un salto condicional puede localizarse y modificarse a nivel de bytes. No se realizaron pruebas contra sistemas ajenos al escenario de práctica ni se intentó vulnerar servicios de terceros.

3. Herramientas y archivos utilizados

- x32dbg, para depurar el ejecutable de 32 bits, ubicar la instrucción de salto y modificarla en tiempo de análisis.
- Python, para automatizar el parche binario de forma reproducible.
- phackeame.exe, ejecutable original proporcionado por el docente.
- phackeame_parcheDg.exe, copia generada al guardar el parche desde x32dbg (Patches -> Save File).



4. Descripción general del escenario

phackeame.exe muestra un formulario de acceso (Usuario / Contraseña, botón "Acceso") y, debajo, un panel "Hack" con un botón "Presioname!". Al revisar las cadenas de texto embebidas en el binario se identificaron referencias a la construcción de un cuerpo JSON (`{"USR": "...", "PASS": "..."}`), a un cliente HTTP y a un servicio de login remoto. Esto indica que la validación de credenciales no ocurre de forma local, sino que el ejecutable envía usuario y contraseña a un servicio web y decide el acceso a partir de la respuesta que recibe.

5. Análisis dinámico con x32dbg

5.1 Carga del ejecutable y ubicación del código de interés

Se abrió phackeame.exe en x32dbg (File -> Open) y se dejó correr con F9 hasta que apareció la ventana del programa.

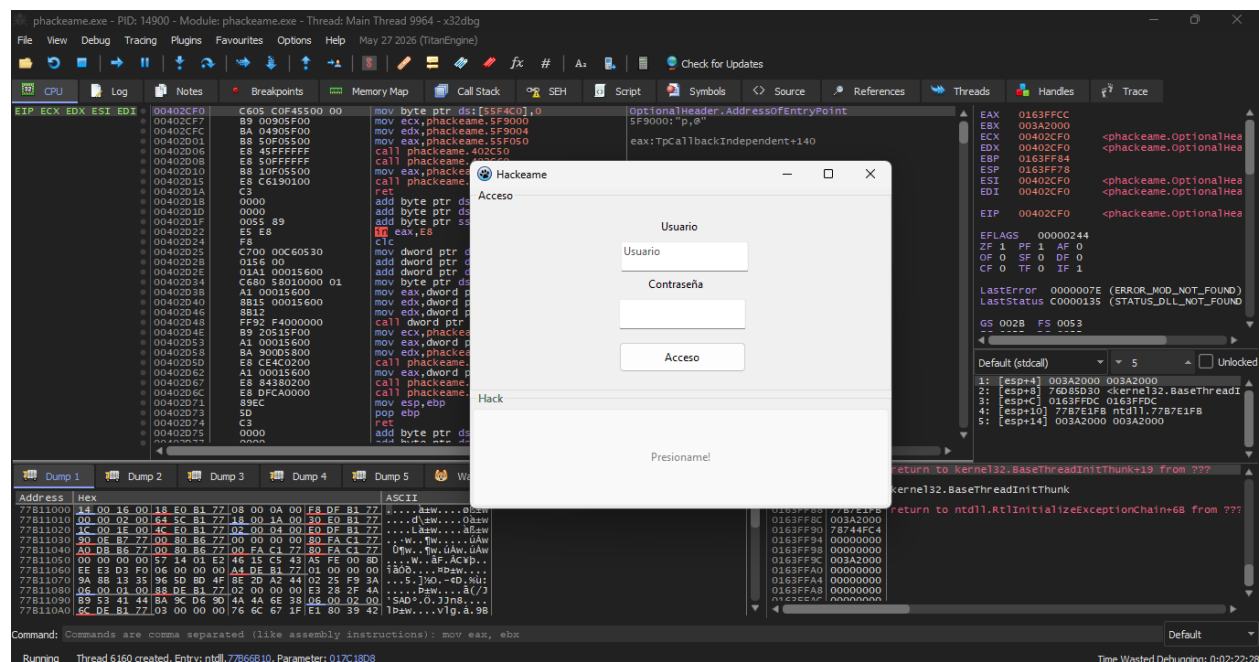


Figura 1. Ventana del programa Hackeame y CPU de x32dbg justo después de cargar phackeame.exe y dejarlo correr.



En lugar de recorrer manualmente todo el ensamblador, se usaron las referencias de cadenas de texto para ir directo al código relevante: clic derecho sobre el panel de desensamblado

-> Search for -> Current Module -> String references.

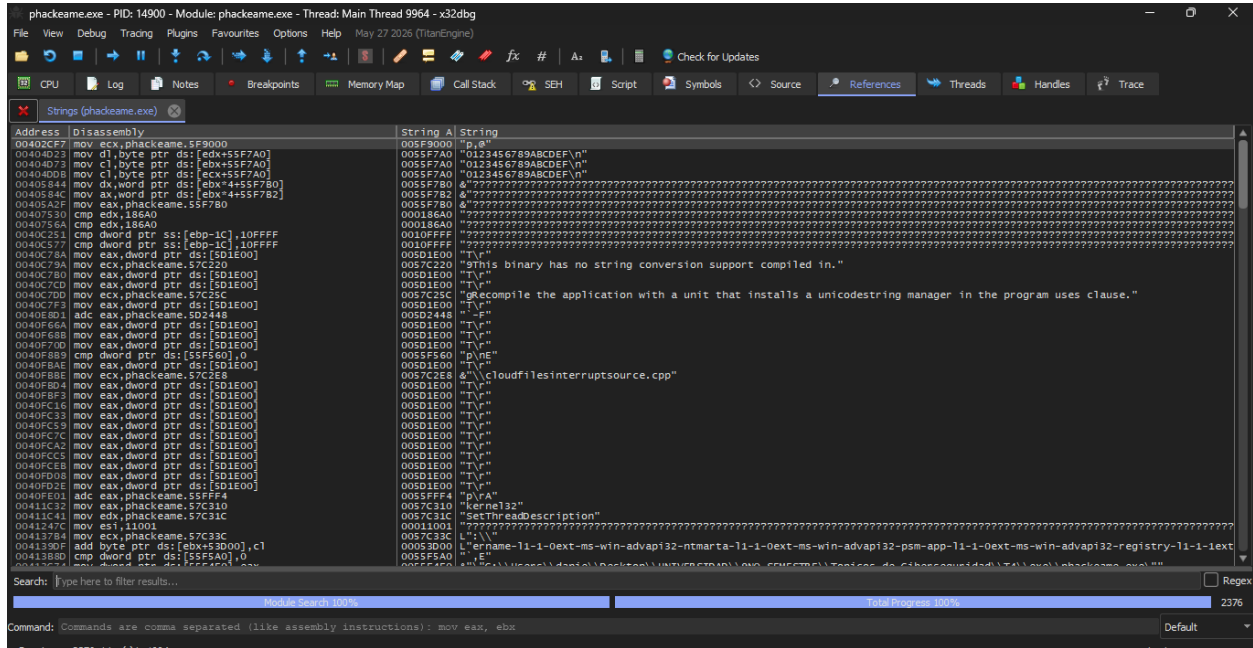


Figura 2. Listado completo de referencias de cadenas de texto (Strings) del módulo phackame.exe.

Filtrando esa lista por "USR" se localizó la cadena {"USR":"-que arma el cuerpo del login- referenciada en la dirección 0042A1A4. Doble clic sobre esa fila regresa a la vista de CPU ya posicionada ahí.

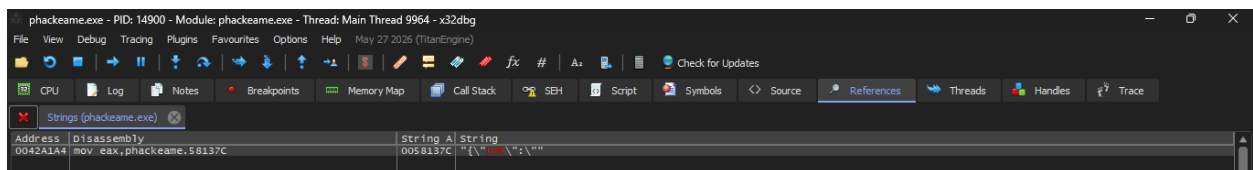


Figura 3. Referencia localizada al filtrar por "USR": la cadena {"USR":" se usa en la dirección 0042A1A4.

5.2 Breakpoint y seguimiento del flujo

Con el cursor sobre 0042A1A4 se colocó un breakpoint (F2). Al avanzar con F8 se observa cómo el programa concatena {"USR":" + usuario + ", "PASS":" + contraseña + "}", arma la URL



del servicio de login y verifica que la petición haya obtenido respuesta (de lo contrario, muestra "No se pudo conectar al servidor").

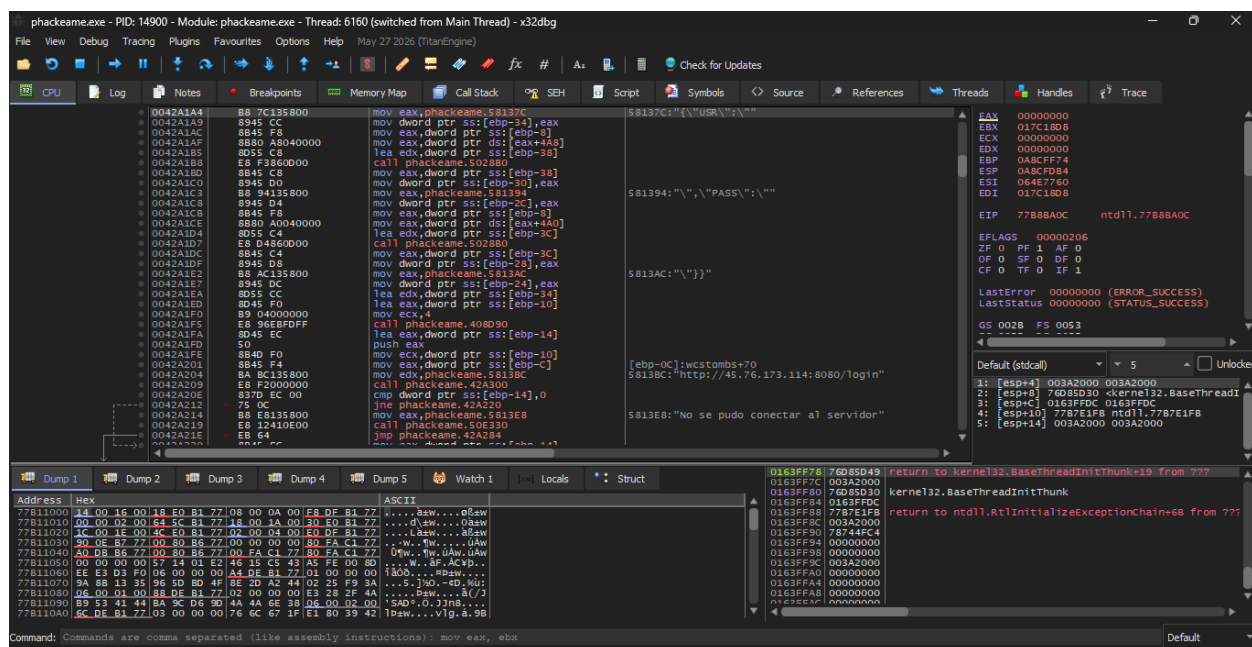


Figura 4. Código en 0042A1A4: construcción del cuerpo JSON con usuario y contraseña, envío por POST y verificación de conexión con el servidor.

Se regresó a la ventana del programa, se introdujo el usuario "Ladrillo" (una credencial inválida, a propósito) y se presionó "Acceso". x32dbg pausó la ejecución justo en el breakpoint colocado en 0042A1A4.

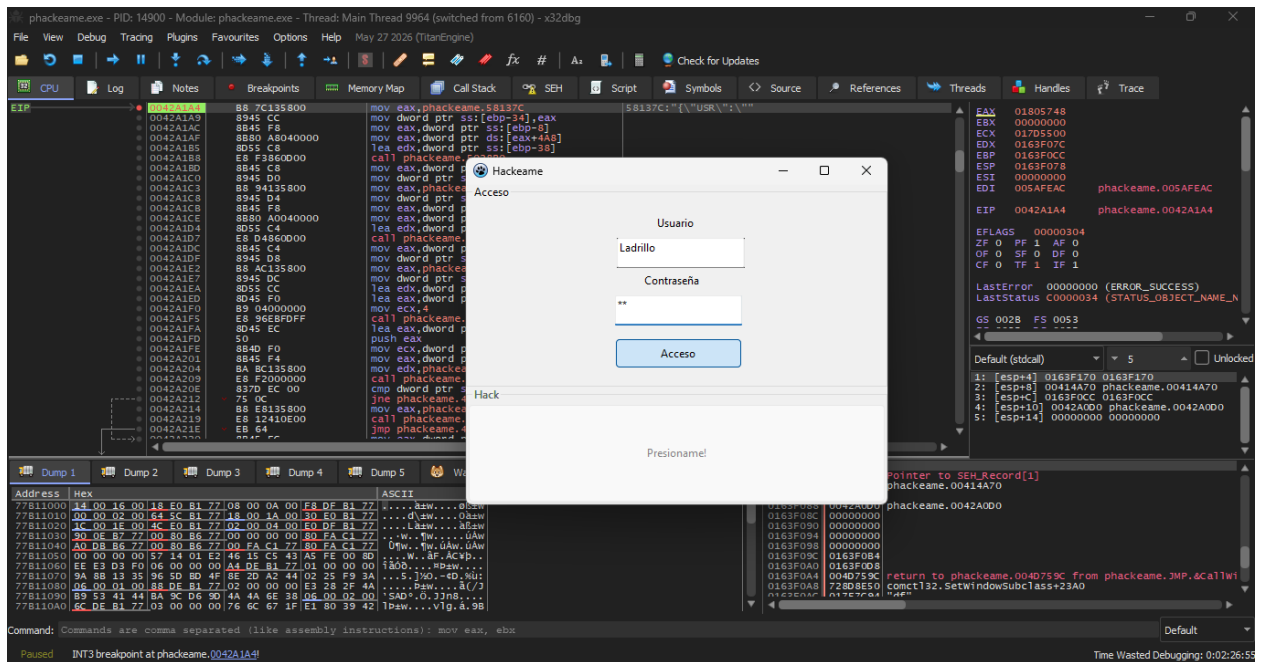


Figura 5. Breakpoint disparado en 0042A1A4 al introducir el usuario "Ladrillo" y presionar "Acceso".

Continuando con F8 se llega al punto donde el programa procesa la respuesta del servidor.

Con una credencial incorrecta, el servicio responde {"R":400}, que el programa procesa antes de decidir si concede o no el acceso.

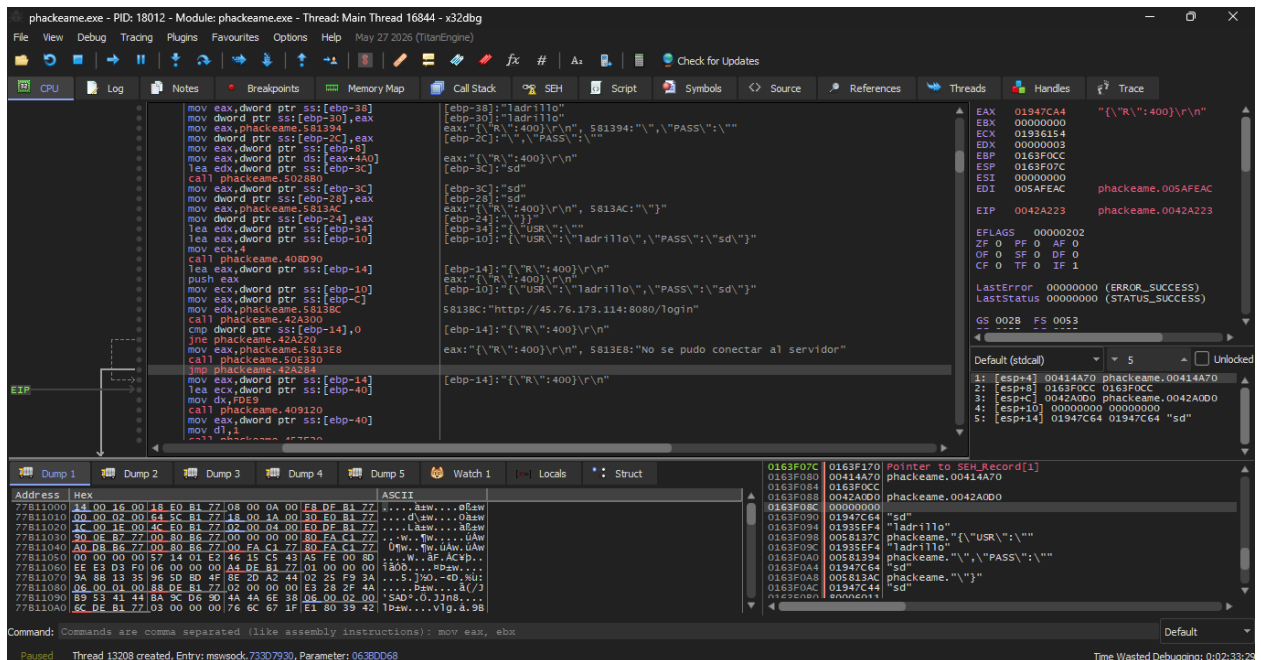


Figura 6. Procesamiento de la respuesta del servidor ({"R":400}) recibida tras enviar una credencial incorrecta.



Nota: durante este recorrido se debe usar F8 (step over) y no F7 (step into) al pasar por las llamadas a funciones internas del programa o de librerías de Windows (por ejemplo shlwapi.dll); usar F7 por error hace que el depurador se meta dentro de esas rutinas, lejos del código que interesa, y basta con Ctrl+F9 ("Execute till return") para salir y retomar el flujo.

5.3 Localización de la instrucción decisiva

Siguiendo con F8 se llega a la comparación que realmente decide el acceso:

```
0042A25F | 81 7D E4 C8 00 00 00 | cmp dword ptr ss:[ebp-1C], 0xC8 ; ¿StatusCode == 200?
0042A266 | 75 1C | jne phackeame.42A284 ; si NO es 200 -> acceso denegado
0042A268 | 8B 45 F8 | mov eax, dword ptr ss:[ebp-8]
0042A271 | B2 01 | mov dl, 1 ; habilita el acceso
0042A27E | FF 91 28 02 00 00 | call dword ptr ds:[ecx+228] ; revela "Lo lograste!"
0042A284 | 31 D2 | xor edx, edx ; salida sin conceder acceso
```

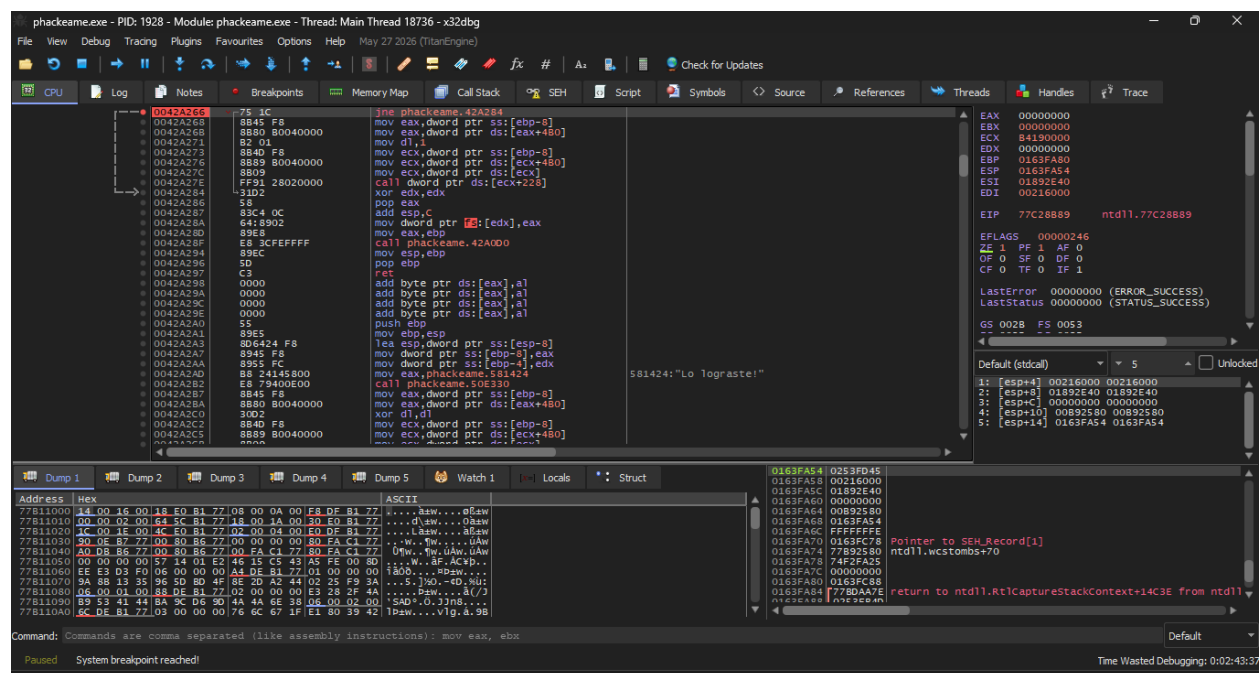


Figura 7. Instrucción decisiva localizada en 0042A25F-0042A266: comparación del StatusCode contra 0xC8 (200) y el salto JNE.

0xC8 equivale a 200 en decimal: el ejecutable compara el StatusCode de la respuesta del servidor contra HTTP 200 OK. Si [ebp-1C] es igual a 200, el JNE no salta y la ejecución continúa



en 0042A268, donde se habilita el acceso. Si el servidor responde cualquier otro código -como ocurrió con la credencial "Ladrillo"- el JNE salta a 0042A284 y el programa nunca concede acceso.

6. El parche

El punto de intervención correcto no es la construcción del JSON ni el envío de la petición HTTP -eso debe seguir funcionando-, sino el único salto condicional que traduce "¿la respuesta fue 200?" en "conceder o negar acceso". Sobre la línea 0042A266 | 75 1C | jne phackeame.42A284 se usó el menú Binary -> Fill with NOPs, indicando un tamaño de 2 bytes (no el valor de 5 que x32dbg propone por defecto, ya que sobrescribiría también el inicio de la rama de acceso concedido).

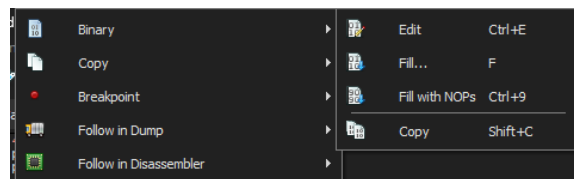


Figura 8. Menú contextual Binary -> Fill with NOPs, usado para neutralizar el salto condicional.

El resultado en la ventana de CPU confirma que únicamente 0042A266 y 0042A267 cambiaron a nop, y que el resto de la función -incluida la rama que revela "Lo lograste!"- permanece intacto:

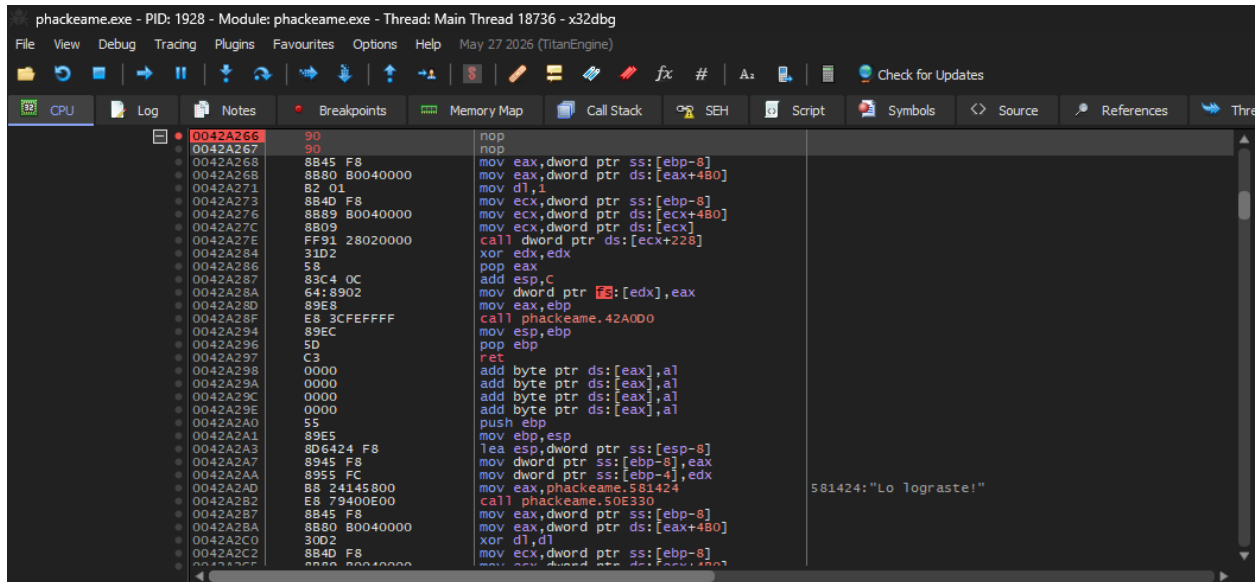


Figura 9. Resultado del parche en memoria: 0042A266 y 0042A267 convertidas en nop, sin alterar el resto de la función.

Con el panel de Patches (Ctrl+P) se verificó que el cambio aplicado fuera exactamente el esperado antes de guardar el archivo: el mensaje "2/2 patch(es) applied!" confirma 2 bytes modificados, ambos en 0042A266-0042A267 (75->90 y 1C->90), sin ningún otro byte alterado.

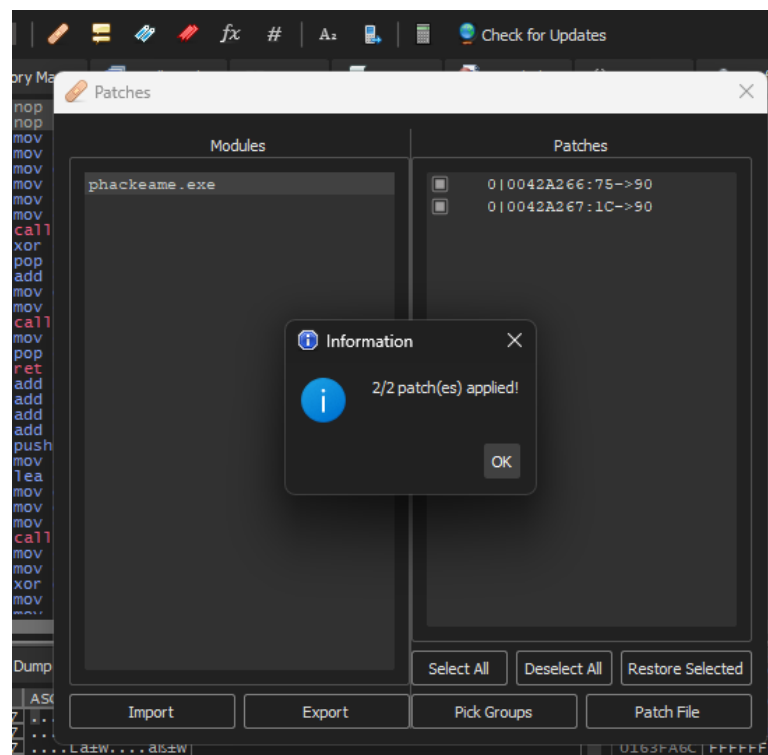




Figura 10. Ventana de Patches confirmando "2/2 patch(es) applied!": únicamente 2 bytes modificados (75->90, 1C->90).

Se usó Save File para generar el ejecutable parchado, guardado como phackeame_parcheDg.exe.

7. Automatización del parche en Python

El mismo cambio se automatizó con un script que localiza el offset de archivo correspondiente a esa instrucción y sustituye sus 2 bytes:

```
from pathlib import Path

RutaOriginal = Path('.') / 'phackeame.exe'
RutaParche = Path('.') / 'phackeame_parche.exe'
Desplazamiento = 0x29666 # offset de archivo del JNE (VA 0x42a266)

if RutaOriginal.exists():
    with open(RutaOriginal, "rb") as original:
        Buffer0 = original.read(Desplazamiento)
        Buffer0 = Buffer0 + b'\x90\x90'
        original.read(2) # saltar los 2 bytes originales del JNE (75 1C)
        Buffer1 = original.read() # todo lo que queda del archivo
        with RutaParche.open("wb") as parche:
            parche.write(Buffer0)
            parche.write(Buffer1)
    print(f"Parche generado: {RutaParche.resolve()}")
else:
    print('No se encontro: ', RutaOriginal.resolve())
```

El punto clave es que Buffer1 = original.read() (sin argumento) debe leer todo lo que resta del archivo después de saltarse el JNE; usar original.read(2) ahí, por ejemplo, truncaría el ejecutable a solo unos cuantos bytes en vez de copiar el resto del binario.

8. Pruebas funcionales

Se cerró la sesión de depuración y se ejecutó phackeame_parcheDg.exe directamente (fuera de x32dbg), usando la misma credencial "Ladrillo" que antes había sido rechazada por el servidor con {"R":400}. Esta vez el programa mostró "Lo lograste!", confirmando que el acceso se concede sin importar el usuario y la contraseña introducidos.

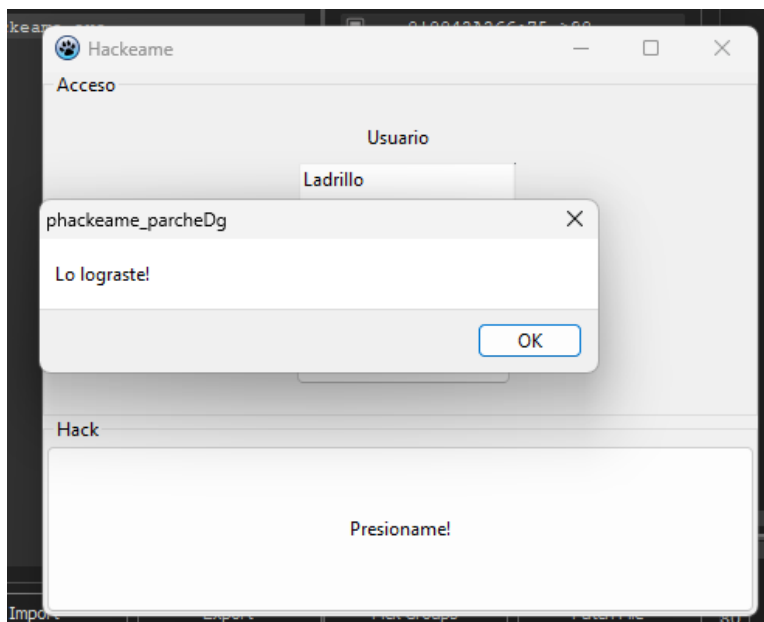


Figura 11. *Prueba funcional: phackeame_parcheDg.exe concede acceso ("Lo lograste!") con la credencial inválida "Ladrillo".*

9. Resultados

- Se identificó el salto condicional relevante en la dirección virtual 0042A266 (offset de archivo 0x29666) dentro de phackeame.exe.
- La decisión de acceso depende de comparar el StatusCode de la respuesta HTTP contra 200 (0xC8) y de un único JNE que separa la rama de acceso concedido de la de acceso denegado.
- El parche sustituye esos 2 bytes (75 1C) por NOP NOP (90 90), neutralizando el salto sin alterar el envío de la petición HTTP ni el resto del binario -confirmado con la ventana de Patches ("2/2 patch(es) applied!").
- Las pruebas funcionales confirman que phackeame_parcheDg.exe concede acceso ("Lo lograste!") incluso con una credencial ("Ladrillo") que el servidor había rechazado explícitamente con {"R":400}.



10. Conclusiones

A diferencia de una validación local (una simple comparación de cadenas), phackeame.exe delega la decisión de acceso a un servicio web y solo revisa si la respuesta trae HTTP 200. Localizar ese punto exigió seguir el flujo completo del manejador del botón "Acceso" en x32dbg -desde la construcción del JSON hasta la extracción del código de estado- para aislar el único salto condicional que realmente importa, evitando distracciones como perderse dentro de rutinas internas de Windows (shlwapi.dll) o aplicar el NOP sobre bytes equivocados por dejar el tamaño por defecto del relleno. Neutralizando esas dos instrucciones (2 bytes) el ejecutable pasa a conceder acceso sin importar el usuario, la contraseña, o lo que responda el servidor de validación, cumpliendo el objetivo de R005.